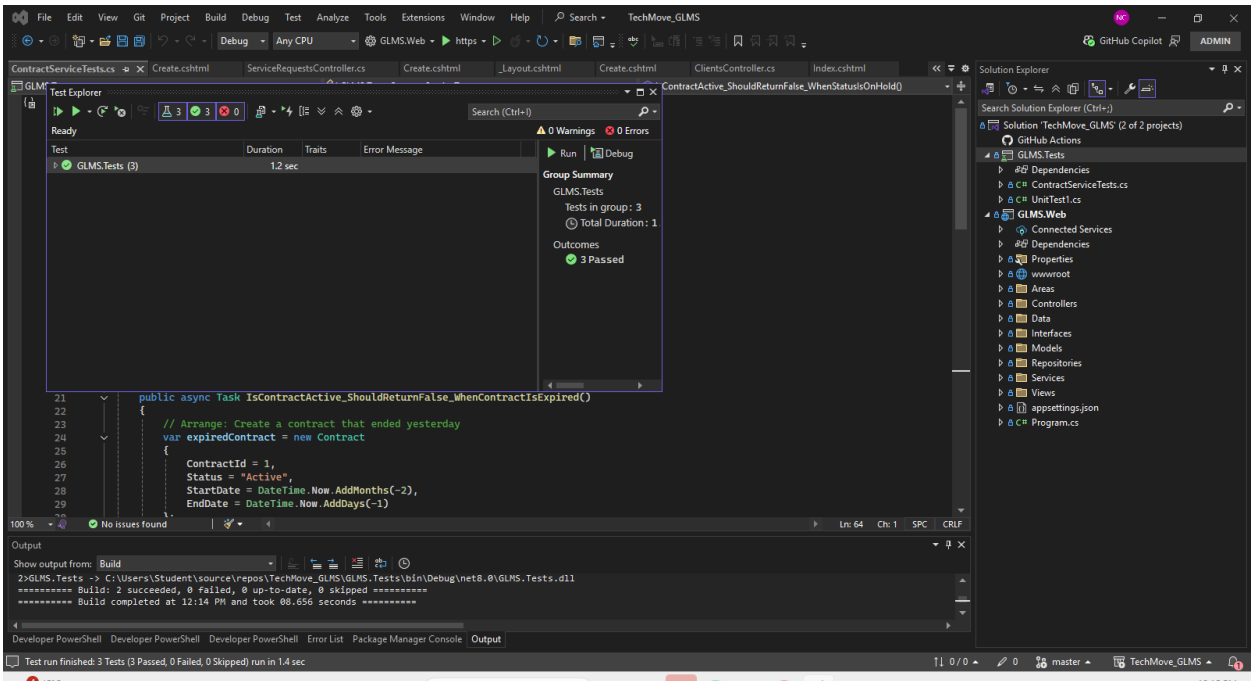
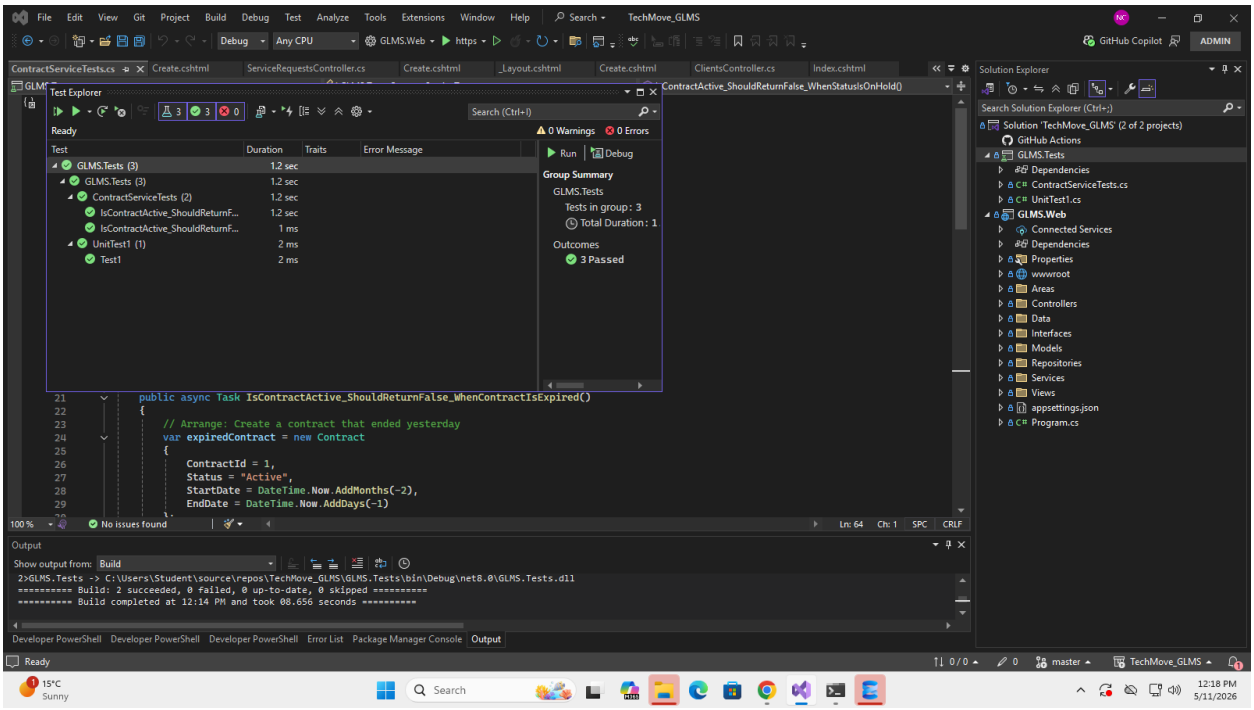
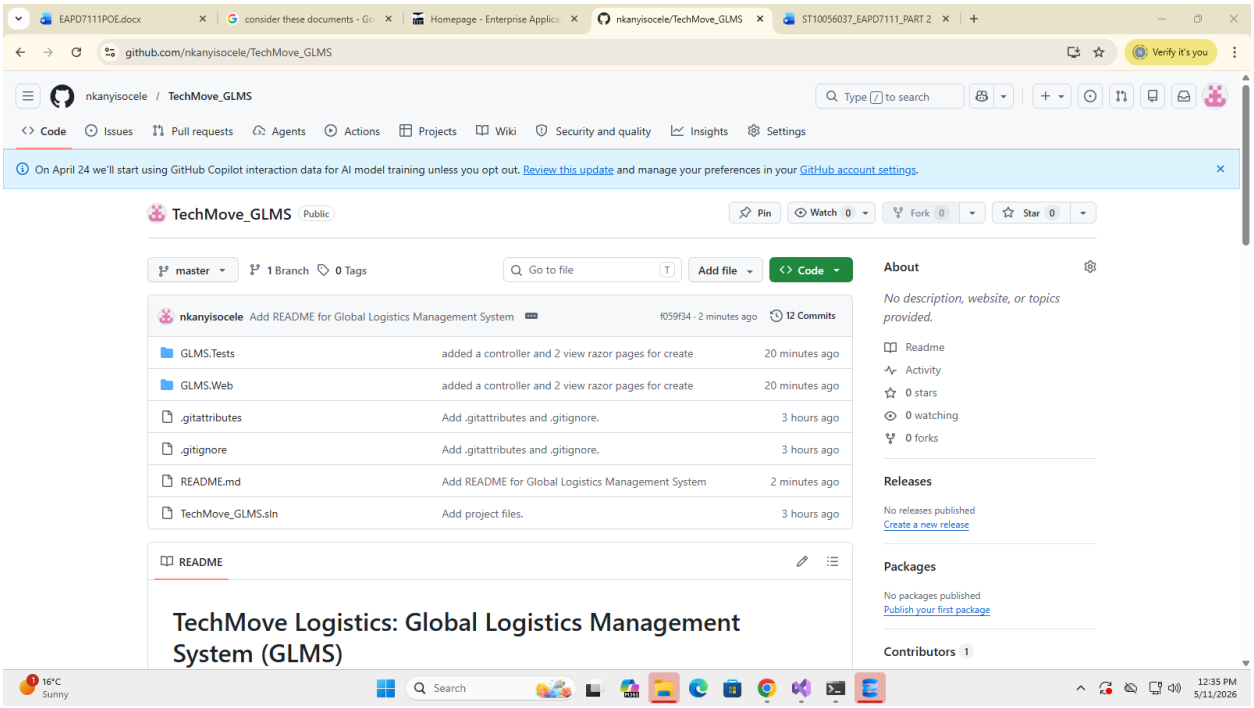
Github: https://github.com/nkanyisocele/TechMove_GLMS

Unit Test Screenshots





Github Screenshot



Technical Documentation: Global Logistics Management System (GLMS)

Company: TechMove Logistics
Architecture: ASP.NET Core 8.0 MVC (Service-Oriented Design)

Executive Summary

The GLMS is a robust, container-ready solution designed to transition TechMove Logistics from manual processes to an automated, scalable platform. The system centralises contract management, enforces strict Service Level Agreement (SLA) validation, and integrates real-time financial data for international operations.

Architectural Overview

To ensure scalability and maintainability, the project follows a **Clean Architecture** approach:

- **Presentation Layer:** ASP.NET Core MVC with Bootstrap for a responsive "Contract Management Hub."

- **Service Layer:** Contains business logic (Currency conversion, File validation, SLA tracking) decoupled from controllers.
- **Data Access Layer:** Implements the **Repository Pattern** and **Dependency Injection** to prevent tightly coupled logic.
- **Database:** SQL Server managed via **Entity Framework Core** with a fully normalized schema.

Rubric Compliance & Feature Highlights

Database Architecture (Rubric Item 1)

- **Normalization:** Implemented a 1-to-Many relationship between Clients, Contracts, and Service Requests.
- **Data Integrity:** Used **Fluent API** in ApplicationDbContext to set decimal precision (18,2) for financial data and enforce unique constraints on client names.
- **Security:** Connection strings are managed via appsettings.json, ensuring no sensitive data is hardcoded.

Design Pattern Implementation (Rubric Item 2)

- **Repository Pattern:** Used to abstract database operations, allowing the application to be "Service-Oriented" and unit-testable.
- **Dependency Injection (DI):** All services and repositories are registered in Program.cs and injected via **Primary Constructors** (modern .NET 8 syntax).
- **Workflow & Validation (Rubric Item 3)**
- **SLA Enforcement:** A dedicated ContractService ensures ServiceRequests can only be logged against contracts that are both chronologically valid (Start/End dates) and marked as "Active."
- **Automated Tracking:** Logic is included to transition contracts to "Expired" automatically based on system date comparisons.

File Handling Mechanism (Rubric Item 4)

- **Robust Storage:** Signed agreements are validated as PDFs and renamed using **UUIDs (GUIDs)** to prevent file collisions.

- **Abstraction:** Handled by a standalone FileService, keeping the Controller clean and focused on routing.

External API Integration (Rubric Item 5)

- **Real-time Finance:** Integrated the **ExchangeRate-API** using IHttpClientFactory.
- **Error Handling:** Implemented try-catch blocks with fallback exchange rates to ensure system uptime during API outages.

Quality Assurance & Testing (Rubric Items 6 & 7)

- **Unit Testing:** Built using **xUnit** to test core business logic in isolation.
- **Mocking:** Utilised the **Moq** library to simulate repository behavior, ensuring tests are fast and do not require a live database.
- **Coverage:** Tests focus on edge cases, such as expired dates and currency math precision.

Deployment & Maintenance

- **Migrations:** Database schema updates are managed via EF Core Migrations for version control.
- **Scalability:** The decoupled service-oriented logic allows for individual components (like the Currency Service) to be moved to Microservices in the future.